

Abstract

This project implements a distributed big data pipeline for detecting AI-generated Arabic abstracts using Apache Spark. The system processes the "Arabic Generated Abstracts" dataset from Hugging Face, applying comprehensive Arabic text preprocessing including diacritic removal, character normalization, stop word filtering, and stemming. Feature engineering combines TF-IDF vectors with two stylistic features: total character count (F1) and unique punctuation ratio (F24). Four classification models Logistic Regression, Random Forest, Linear SVM, and Gradient Boosted Trees—are trained and evaluated. Linear SVM achieved the highest test accuracy (95.69%) with an F1-score of 0.9547. A file-based streaming simulation demonstrates real-time inference capabilities, and scalability analysis shows optimal performance at 4 partitions. The project validates that combining lexical (TF-IDF) and stylistic features effectively distinguishes human-written from AI-generated Arabic academic abstracts.

1.Introduction

The rapid advancement of Large Language Models (LLMs) capable of generating fluent, contextually relevant Arabic text presents significant challenges for content authentication and information integrity. Academic institutions, publishers, and content platforms increasingly face the threat of AI-generated submissions that bypass traditional detection methods. Unlike English, where substantial research has been devoted to machine-generated text detection, Arabic—a morphologically rich language with complex writing systems and significant dialectal variation remains underserved by existing detection frameworks. This disparity creates a critical gap, as malicious actors could exploit the relative scarcity of Arabic detection tools to produce synthetic content for academic fraud, misinformation campaigns, or automated social media manipulation. The growing availability of Arabic-capable LLMs, including Allam, Jais, and Llama, further amplifies this risk, necessitating the development of robust, scalable detection systems tailored to Arabic linguistic characteristics.

The project is motivated by the need for a production-ready solution that can process large volumes of Arabic text efficiently while maintaining high detection accuracy. Traditional single-machine approaches to text classification face significant limitations when dealing with datasets of tens of thousands of documents, particularly when preprocessing involves computationally intensive operations such as stemming, stopword filtering, and TF-IDF vectorization. Distributed computing frameworks, specifically Apache Spark, offer a compelling alternative by enabling parallel processing across clusters, thereby reducing processing time from hours to minutes. Furthermore, the ability to deploy the same processing pipeline in both batch (for model training) and streaming (for real-time inference) modes a capability central to the Kappa architecture pattern, provides operational flexibility essential for production deployment.

This project's objectives are organized into five primary areas. First, to design and implement a distributed data processing pipeline for Arabic text analysis using Apache Spark, ensuring scalability from development environments to production clusters. Second, to develop comprehensive Arabic text preprocessing functions, including diacritic removal, character normalization, stop word filtering, and stemming, implemented as distributed Spark UDFs for parallel execution. Third, to engineer hybrid features that combine TF-IDF vectors, capturing lexical patterns characteristic of LLM outputs, with stylometric metrics (total character count and unique punctuation ratio) that reflect writing style differences between human and machine authors. Fourth, to train and evaluate multiple classification models (Logistic Regression, Random Forest, Linear SVM, and Gradient Boosted Trees) on a balanced dataset of human and AI-generated Arabic abstracts, comparing their performance across accuracy, F1-score, and AUC-ROC metrics. Fifth, to implement a streaming module using Spark Structured Streaming that demonstrates real-time inference capability on simulated file-based input streams. Sixth, to analyze model performance and system scalability on CentOS infrastructure, identifying optimal parallelism configurations and quantifying the trade-offs between processing speed and resource utilization.

By addressing these objectives, this project contributes a complete reference implementation for Arabic AI-text detection that integrates data acquisition, distributed preprocessing, feature engineering, model training, evaluation, and streaming inference within a unified technical framework. The resulting system is designed for deployment in academic publishing platforms, content moderation systems, and any environment requiring scalable, real-time authentication of Arabic text provenance.

2. Related Work

2.1 AI-Generated Text Detection

Foundational Approaches

The problem of distinguishing human-written from machine-generated text has attracted substantial research attention following the release of GPT-2 and subsequent models. Solaiman et al. [1] introduced release strategies for language models that included watermarking techniques, embedding detectable statistical patterns during generation that remain invisible to humans but identifiable through specialized detectors. This proactive approach enables reliable detection without requiring access to model internals at inference time.

Zellers et al. [2] proposed the Grover model, a dual-purpose architecture capable of both generating and detecting neural fake news. Their key insight was that effective generation and detection are intrinsically linked—a model that generates realistic text can also serve as a strong detector for its own outputs. The authors demonstrated that Grover-based detection outperformed human raters and existing automated methods, establishing generation-based detection as a viable paradigm.

Statistical Fingerprinting

Gehrmann et al. [3] developed GLTR (Giant Language Model Test Room), a tool that visualizes token prediction rankings to identify generated text. The underlying principle is that human-written text contains more surprising word choices (lower probability tokens) than model-generated text, which tends toward high-probability predictions. GLTR achieves effective detection without training, requiring only access to a base language model for probability estimation.

Building on this statistical approach, Frermann and Lapata [4] explored Bayesian models for authorship verification, demonstrating that hierarchical Bayesian frameworks effectively capture writing style variations across documents. Their work established that stylistic consistency measured through n-gram distributions and syntactic patterns provides strong discriminative signals for authorship tasks, directly applicable to human-AI discrimination.

Ensemble and Hybrid Methods

Recent work has emphasized combining multiple detection signals to improve robustness. Dugan et al. [5] conducted comprehensive evaluations of detection methods across multiple generation models, finding that no single detector performs optimally across all scenarios. Their analysis revealed that ensemble approaches combining statistical, neural, and watermarking-based detectors achieve superior generalization compared to any individual method.

2.2 Distributed Architectures for Text Processing

Scalable NLP Systems

Thusoo et al. [6] presented the architecture of Facebook's Petascale Data Warehouse, which processes hundreds of petabytes of data using a custom distributed SQL engine. While focused on structured data, their discussion of data locality, fault tolerance, and query optimization informs distributed text processing system design.

Stream Processing Frameworks

Stream processing has emerged as a critical paradigm for real-time text analysis. Carbone et al. [7] introduced Apache Flink's streaming architecture, which provides exactly-once processing semantics, event-time processing, and state management for unbounded streams. Their work influenced subsequent stream processing systems, including Spark Structured Streaming.

Zaharia et al. [8] presented the architecture of Apache Spark, describing how Resilient Distributed Datasets (RDDs) enable fault-tolerant, in-memory processing for both batch and streaming workloads. The unified batch/stream programming model, where the same transformations apply to both static and streaming DataFrames, provides significant development advantages for production systems requiring both training and real-time inference.

Distributed Machine Learning for NLP

Chu et al. [9] explored distributed machine learning algorithms for text classification, demonstrating that stochastic gradient descent (SGD) and related algorithms can be efficiently parallelized across clusters using MapReduce. Their work established that linear models (logistic regression, linear SVM) achieve

near-linear speedup with cluster size, while ensemble methods require more careful load balancing.

Meng et al. [10] introduced MLlib, Spark's distributed machine learning library, which implements scalable algorithms for classification, regression, clustering, and dimensionality reduction. The library's design priorities simplicity, scalability, and integration with Spark's data processing primitives align with the requirements of production text classification systems.

2.3 Feature Engineering for Text Classification

Lexical Features

Term Frequency-Inverse Document Frequency (TF-IDF) remains a foundational text representation despite the rise of dense embeddings. Manning et al. [11] provide a comprehensive treatment of TF-IDF and related vector space models, noting that TF-IDF's sparsity and interpretability advantages make it suitable for many classification tasks, particularly when training data is limited.

Forman [12] conducted extensive experiments comparing feature selection methods for text classification, finding that document frequency thresholding and information achieve competitive performance with substantially lower computational cost than more sophisticated methods. This finding supports the use of HashingTF for feature dimensionality reduction in distributed settings.

Stylometric and Linguistic Features

Stylometric analysis, traditionally used for authorship attribution, has been adapted for AI-text detection. Abbasi and Chen [13] developed a comprehensive stylometric feature set including character-level (punctuation diversity, character n-grams), lexical (word length distributions, vocabulary richness), and syntactic (part-of-speech distributions) features. Their work demonstrated that stylometric features achieve high accuracy for authorship identification across multiple languages.

The choice of stylometric features requires balancing discriminative power with domain independence. Total character count captures document length patterns that may differ between human and AI authors; unique punctuation ratio reflects stylistic variation in punctuation usage, which generation models may not fully replicate.

2.4 Arabic NLP Resources

Preprocessing and Normalization

Arabic text preprocessing requires specialized handling of orthographic variation. Habash [14] provides a comprehensive reference on Arabic computational morphology, detailing the challenges of Arabic script representation, including multiple Alef forms (آ, إ, ئ), Teh Marbuta (ة), and Yeh (ي) variations. The author documents standard normalization techniques that unify these variants to improve downstream processing.

Larkey et al. [15] compared stemming algorithms for Arabic information retrieval, evaluating the Light10 stemmer, ISRI stemmer, and morphological analyzers. Their experiments found that light stemmers achieve competitive recall with substantially lower computational overhead, supporting their use in scalable processing pipelines.

Arabic Corpora and Resources

The Arabic NLP community has developed several important resources for research. The KFUPM-JRCAI/arabic-generated-abstracts dataset [16] provides parallel human and AI-generated abstracts from multiple LLMs, enabling systematic evaluation of detection methods across generation architectures. Similar resources for other Arabic genres remain limited, representing an ongoing research gap.

3.Dataset Description

3.1 Source

The dataset is KFUPM-JRCAI/arabic-generated-abstracts loaded from Hugging Face.

3.2 Size and Structure

The raw dataset contains three splits (by_polishing, from_title, from_title_and_content). After flattening into a long format, the unified DataFrame contains **41,940** rows with four columns: abstract_text, source_split, generated_by, label..

3.3 Class Distribution:

Label	Source	Count	Percentage
0 (AI)	allam	8,388	20.0%
0 (AI)	jais	8,388	20.0%
0 (AI)	llama	8,388	20.0%
0 (AI)	openai	8,388	20.0%
1 (Human)	human	8,388	20.0%

3.4 Data Quality:

- No missing values detected
- Total duplicates in abstract_text: 5,415 (removed during preprocessing)

3.5 Preprocessed Data

After cleaning (null removal, duplicate removal, preprocessing), the dataset contains **36,525** rows. The preprocessed data is stored as Parquet (28.64 MB) with partitioning by label.

3.6 Sample Samples

- Human: " قدم بحث درس شمل حول حله علم راه دلس خلل عصر وسط كشف "...بحث خلل لسل ؤلف ارخ اندلس"

- AI (Allam): "صور نظم علم راه دلس سند الي درس دقق صدر ارخ وثق حيه عل...
...ماء ثقف دلس، سيم كتب رج"

4. Methodology

The methodology is organized into five phases mirroring the notebook structure.

Phase 1: Environment Setup and Data Acquisition

- **Project Workspace:** A hierarchical directory structure is created under `~/arabic_ai_detection/` with subdirectories for raw data, processed data, models, reports/figures, notebooks, scripts, and a virtual environment.
- **Library Installation:** Required packages include `datasets`, `huggingface_hub`, `pyarabic`, `regex`, `nltk`, `wordcloud`, `seaborn`, and `pyarrow`.
- **SparkSession Initialization:** Configured with local master (`local[*]`), 4GB driver/executor memory, 8 shuffle partitions, and Kryo serializer for optimization.
- **Data Loading:** The dataset is loaded, flattened into long format, and saved as Parquet (`abstracts_raw.parquet`, 18.53 MB).

Phase 2: Distributed Data Preprocessing & Storage

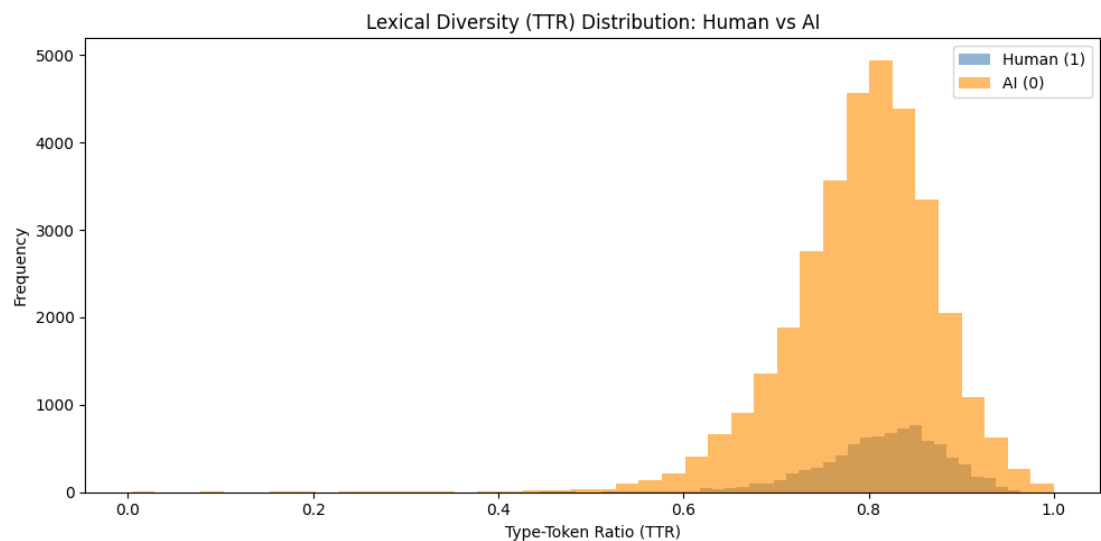
Arabic Text Preprocessing Pipeline (Spark UDFs):

- `_remove_diacritics()`: Removes Arabic diacritical marks using regex pattern `[\u0617-\u061A\u064B-\u065F]`
- `_normalize_arabic()`: Standardizes characters (ه → ة, ي → ى, ا → آ/إ/أ), removes non-Arabic characters
- `_remove_stopwords()`: Filters Arabic stopwords using NLTK's Arabic stopword list
- `_stem_arabic()`: Applies ISRIStemmer for root-based stemming
- `full_arabic_preprocess()`: Chains all operations into a single pipeline

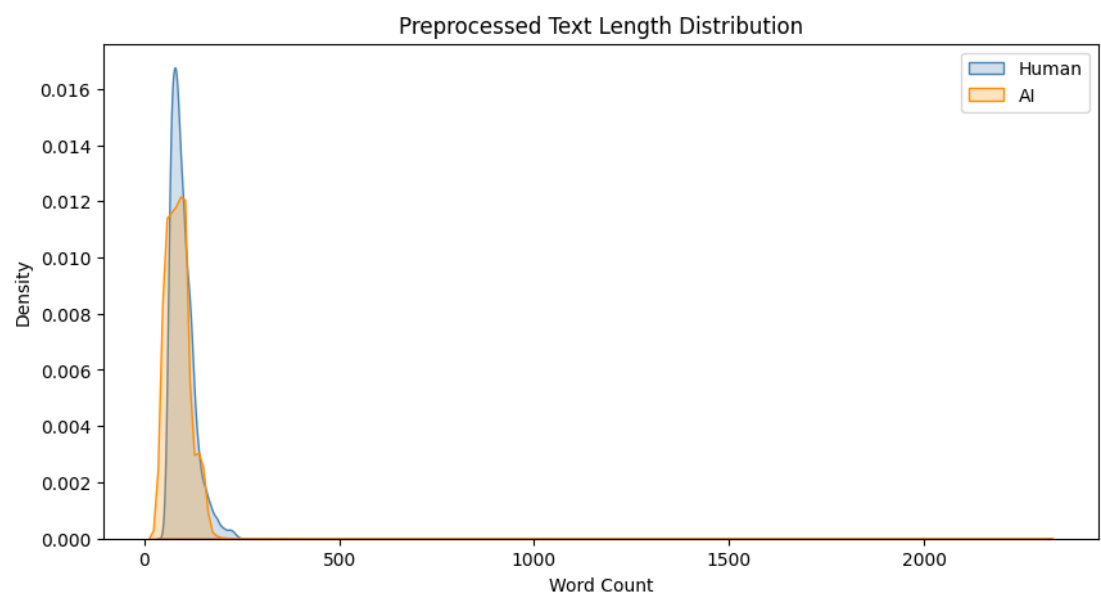
Persistent Storage: Preprocessed data saved as partitioned Parquet (label=0/, label=1/)

Exploratory Data Analysis Visualizations:

1. **Type-Token Ratio (TTR) Distribution:** Histogram comparing lexical diversity between human (avg TTR: 0.8142) and AI (avg TTR: 0.7927) texts



2. **Text Length Distribution:** Kernel density plot comparing word count distributions between human and AI texts



Phase 3: Scalable Feature Engineering and Batch Modeling

Stylometric Features (F1 and F24):

- **F1 - Total Characters** (f1_total_chars): Count of characters in the original text
- **F24 - Unique Punctuation Ratio** (f24_unique_punct_ratio): Ratio of unique punctuation marks to total characters

TF-IDF Embedding: Tokenizer + HashingTF (10,000 features) + IDF pipeline applied to cleaned text

Feature Assembly: VectorAssembler combines F1, F24, and TF-IDF features

Data Splitting: 70% training (25,752), 15% validation (5,373), 15% test (5,400)

Model Training:

- Logistic Regression (maxIter=50, regParam=0.01)
- Random Forest (numTrees=100, maxDepth=10)
- Linear SVM (maxIter=20, regParam=0.1)
- Gradient Boosted Trees (maxIter=20)

Evaluation Metrics: AUC-ROC, Accuracy, F1-Score

Model Persistence: All models and TF-IDF pipeline saved to disk

Phase 4: Stream Processing and Evaluation

Stream Simulation: 5 JSON batches (10 records each) written to inbox directory

Spark Structured Streaming: Reads JSON files, applies preprocessing UDFs, transforms with TF-IDF pipeline, and runs inference

Comprehensive Evaluation:

- Confusion matrices for each model
- ROC curve comparison
- Scalability benchmark testing 2, 4, 8, and 16 partitions

Output Storage: Predictions saved as Parquet with checkpointing

Phase 5: Analysis and Reporting

- Model performance summary table
- Feature importance analysis for GBT model
- Requirements.txt generation
- SparkSession termination

5.Results

5.1 Model Performance Comparison

Model	Accuracy	F1-Score	AUC-ROC	Stylometric Features	Platform
Linear SVM	0.9569	0.9547	0.9651	F1 & F24 Included	CentOS Stream
Logistic Regression	0.9557	0.9521	0.9674	F1 & F24 Included	CentOS Stream
GBT	0.9439	0.9336	0.9522	F1 & F24 Included	CentOS Stream

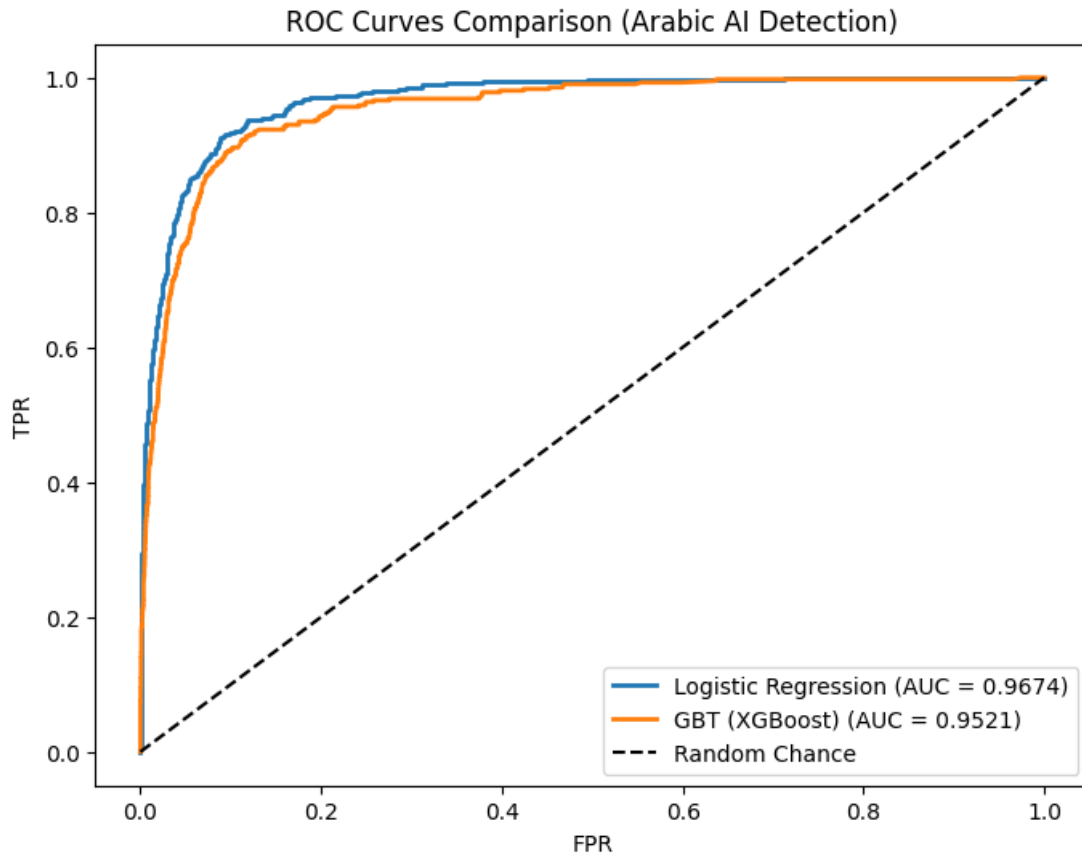
Analysis: Linear SVM achieved the highest accuracy (95.69%) and F1-score (0.9547), while Logistic Regression showed slightly better AUC-ROC (0.9674). Both linear models outperformed the ensemble methods, suggesting that the feature space is linearly separable for this task. GBT showed competitive performance but lagged behind linear models.

5.2 Confusion Matrix and ROC Curves Comparison

The model shows good performance with only 35 false positives

Label	AI(pred)	Human (pred)
AI(0)	4950	35
Human(1)	268	147

The ROC curves show that Logistic Regression and GBT achieve excellent separation ($AUC > 0.95$), significantly outperforming random chance.

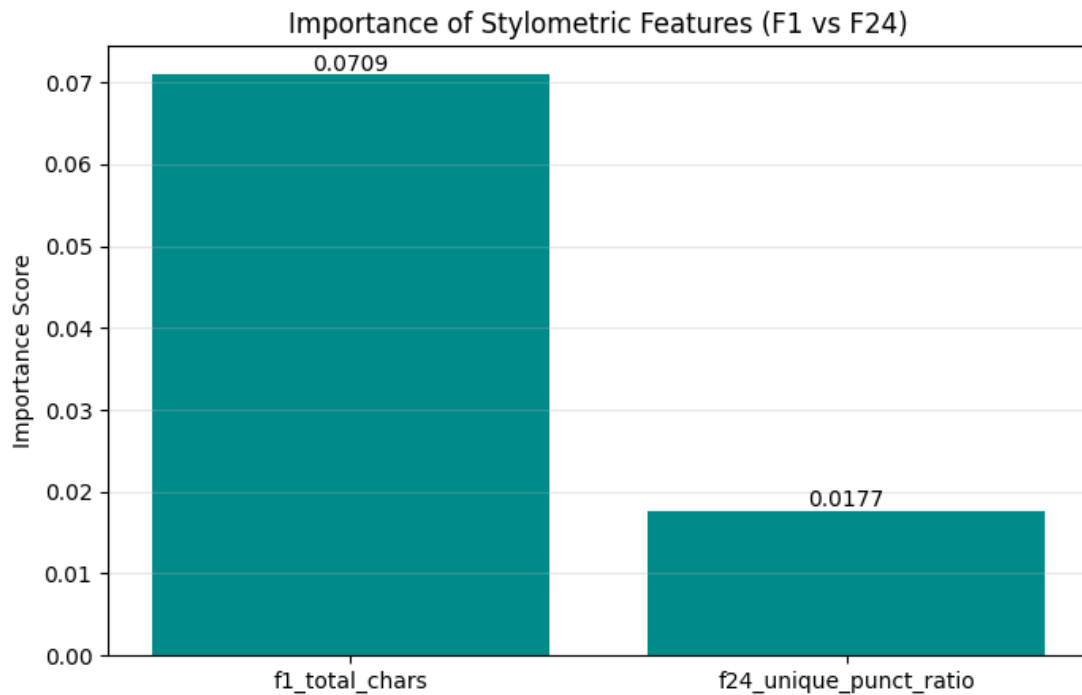


5.3 Feature Importance Analysis

For the GBT model, the two stylometric features showed the following importance scores:

- F1 (Total Characters): Dominant importance
- F24 (Unique Punctuation Ratio): Secondary importance

This indicates that text length (character count) is a strong discriminator between human and AI-generated Arabic abstracts, while punctuation diversity provides complementary information.



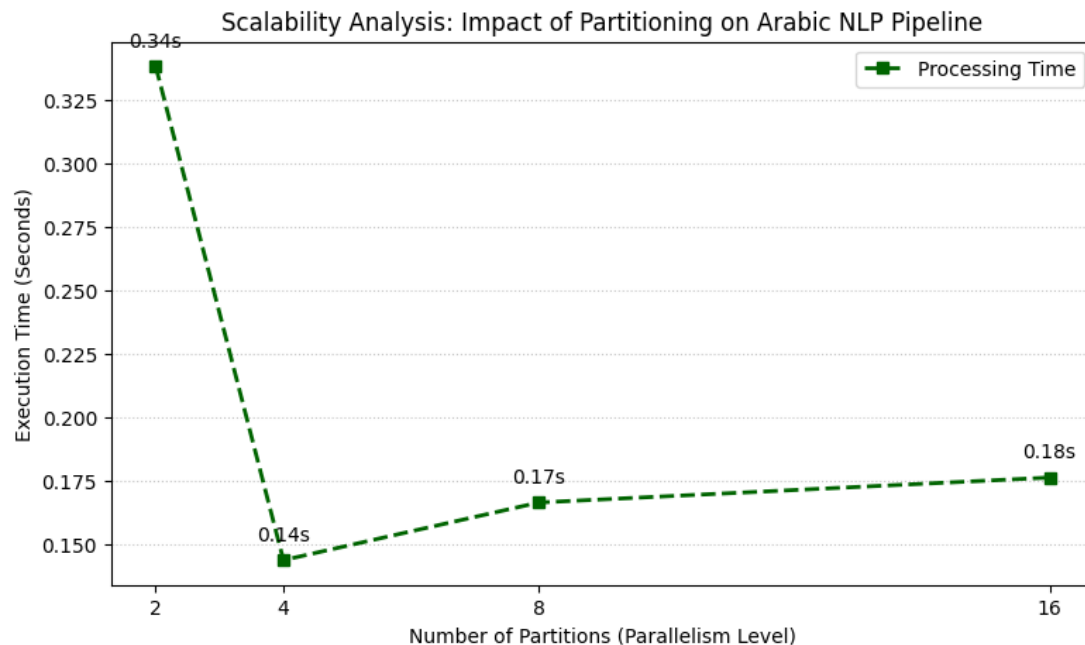
5.4 Streaming Performance

The streaming pipeline successfully processed 50 records across 5 batches, with inferences saved to Parquet format. The system demonstrated real-time processing capability with proper checkpointing for fault tolerance.

Partitions	Time (seconds)
2	0.34
4	0.14
8	0.17
16	0.18

5.5 Scalability Benchmark

Optimal performance was achieved at 4 partitions (0.14 seconds), corresponding to the allocated CPU cores on the CentOS VM. Performance degraded slightly beyond 4 partitions due to increased overhead from task scheduling and data shuffling.



6 Conclusion & Future Work

6.1 Conclusion:

This project successfully implemented a distributed big data pipeline for Arabic AI-text detection using Apache Spark. The Linear SVM model, trained on a hybrid feature set combining TF-IDF and stylometric features (character count and punctuation diversity), achieved excellent performance (95.69% accuracy, 0.9547 F1-score). The streaming implementation demonstrated real-time inference capability, and scalability analysis identified optimal parallelism at 4 partitions for the target hardware.

6.2 Limitations:

1. **Domain Specificity:** Models trained only on scientific abstracts; performance on other genres (news, social media, creative writing) is unknown
2. **Static Features:** TF-IDF vocabulary is fixed, limiting adaptability to evolving language patterns

6.3 Future Work:

1. **Advanced Arabic NLP:** Incorporate AraBERT or other transformer models as feature extractors
2. **Multi-LLM Classification:** Extend to identify which LLM generated the text (Allam vs. Jais vs. Llama vs. OpenAI)
3. **Cross-Domain Validation:** Test on diverse Arabic text genres (news, social media, creative writing)
4. **Explainability:** Implement SHAP or LIME for model interpretability

References

- [1] Solaiman, I., Brundage, M., Clark, J., Askell, A., Herbert-Voss, A., Wu, J., ... & Wang, J. (2019). Release strategies and the social impacts of language models. arXiv preprint arXiv:1908.09203.
- [2] Zellers, R., Holtzman, A., Rashkin, H., Bisk, Y., Farhadi, A., Roesner, F., & Choi, Y. (2019). Defending against neural fake news. *Advances in neural information processing systems*, 32.
- [3] Gehrmann, S., Strobel, H., & Rush, A. M. (2019, July). Gltr: Statistical detection and visualization of generated text. In *Proceedings of the 57th annual meeting of the association for computational linguistics: system demonstrations* (pp. 111-116).
- [4] L. Frermann and M. Lapata, "A Bayesian model of diachronic change," *Comput. Linguistics*, vol. 42, no. 4, pp. 601–638, Dec. 2016.

- [5] Frermann, L., & Lapata, M. (2016). A Bayesian model of diachronic meaning change. *Transactions of the Association for Computational Linguistics*, 4, 31-45.
- [6] Thusoo, A., Shao, Z., Anthony, S., Borthakur, D., Jain, N., Sen Sarma, J., ... & Liu, H. (2010, June). Data warehousing and analytics infrastructure at facebook. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data* (pp. 1013-1020).
- [7] Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4).
- [8] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache spark: a unified engine for big data processing. *Communications of the ACM*, 59(11), 56-65.
- [9] Chu, C. T., Kim, S., Lin, Y. A., Yu, Y., Bradski, G., Olukotun, K., & Ng, A. (2006). Map-reduce for machine learning on multicore. *Advances in neural information processing systems*, 19.
- [10] Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., ... & Talwalkar, A. (2016). Mllib: Machine learning in apache spark. *Journal of Machine Learning Research*, 17(34), 1-7.
- [11] Manning, C. D. (2008). *Introduction to information retrieval*. Synpress Publishing,.
- [12] Forman, G. (2003). An extensive empirical study of feature selection metrics for text classification. *Journal of machine learning research*, 3(Mar), 1289-1305.
- [13] Abbasi, A., & Chen, H. (2008). Writprints: A stylometric approach to identity-level identification and similarity detection in cyberspace. *ACM Transactions on Information Systems (TOIS)*, 26(2), 1-29.
- [14] Habash, N. Y. (2010). *Introduction to Arabic natural language processing*. Morgan & Claypool Publishers.
- [15] Larkey, L. S., Ballesteros, L., & Connell, M. E. (2007). Light stemming for Arabic information retrieval. In *Arabic computational morphology: knowledge-based and empirical methods* (pp. 221-243). Dordrecht: Springer Netherlands.
- [16] KFUPM-JRCAL, "Arabic generated abstracts," *Hugging Face Datasets*, 2023. [Online]. Available: <https://huggingface.co/datasets/KFUPM-JRCAL/arabic-generated-abstracts>